A Toolkit for High Performance Event Driven Systems

Ben Gangl-Lipson

Table of Contents

1. Schema Validation
2. Provenance
3. Fact Events
4. Hashing
5. Compaction
6. Batching and Timeouts
7. Deduplication
8. Fresh Filtering
9. Draining
10. Caching
11. Concurrency
12. Compression

Schema Validation

- Introduce strict typing into Kafka
- Typically uses an external schema registry
- Serialization and deserialization via schema ID
- Key and value are separate validations
- Broker schema validation vs producer schema validation

Schema Validation

Benefits

1. Strong typing and contracts
2. Safe version control
3. Better observability
4. Strong documentation

Drawbacks

1. Operational complexity running a registry
2. Slower prototyping
3. Conventions must be agreed upon

Schema Validation

When and how can I use it?

When?

- Almost always
- Topics are consumed publicly across the enterprise

How?

- Schema registry like Confluent Schema Registry
- Serialization format like Avro, Protobuf, or JSON
- `key.serializer` and `value.serializer` config values
- `schema.registry.url` for producer validation and `confluent.value.schema.validation` for broker validation

```
val schemaStr = """
    {
      "type": "record",
      "name": "UserCreated",
      "namespace": "com.example.avro",
      "fields": [
        {"name":"id","type":"string"},
        {"name":"email","type":"string"},
        {"name":"phone","type":["null","string"], "default": null}
      ]
    }
""".trimIndent()

val schema = Schema.Parser().parse( s = schemaStr)

val props = Properties().apply {
    put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092")
    put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, "org.apache.kafka.common.serialization.StringSerializer")
    put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, "io.confluent.kafka.serializers.KafkaAvroSerializer")
    put("schema.registry.url", "http://localhost:8081")
    put("auto.register.schemas", "false")
}

val producer = KafkaProducer<String, GenericRecord>( properties = props)

val rec = GenericData.Record(schema).apply {
    put("id", "user-100")
    put("email", "ben@example.com")
    put("phone", null)
}

val key = rec.get("id").toString()
val record: ProducerRecord<String?, GenericRecord?> = ProducerRecord(topic, key, value = rec)

try {
    val future = producer.send(record) { metadata, exception ->
        if (exception != null) {
            println("Async send failed: ${exception.javaClass.simpleName}: ${exception.message}")
            val cause = exception.cause
            if (cause != null) {
                when (cause) {
                    is RestClientException -> {
                        println("Schema registry error: status=${cause.status}, message=${cause.message}")
                    }

                    else -> {
                        println("Send path cause: ${cause.javaClass.name}: ${cause.message}")
                    }
                }
            }
        } else {
            println("Produce succeeded: topic=${metadata.topic()}, partition=${metadata.partition()}, offset=${metadata.offset()}")
        }
    }
    future.get()
} catch (serEx: SerializationException) {
```

Provenance

- Metadata attached to an event to identify it across systems
- Troubleshooting and explaining data
- Typically set in the headers
- All inputs get set on the output
- Very few drawbacks if any

Provenance

When and how can I use it?

When?

- Always set a unique ID to events
- Forward those same IDs when filtering data
- Add those unique IDs to the body when transforming an input topic or aggregating multiple input topics

How?

- Kafka headers
- Random UUIDs with timestamps and source topic names

```
data class Provenance(
    val spanId: String? = null,
    val timestamp: Instant,
    val entity: String,
)

fun generateSpanId(): String = UUID.randomUUID().toString()
```

```
fun buildProvenance(): Provenance = Provenance(
    spanId = headers[SPAN_ID_HEADER_NAME]?.let { String( bytes = it) },
    timestamp = Instant.ofEpochMilli(timestamp),
    entity = "kafka://${formatBrokerProvenance(broker)}/$topic/$partition/$offset",
)
```

```
outboundMessage.messages.forEach { transformerMessage ->
    if (transformerMessage.kafkaOutputMessage.provenances.isEmpty()) {
        transformerMessage.kafkaOutputMessage.provenances.add(kafkaMessage.buildProvenance())
    }
}
```

Fact Events

- Alternative to deltas
- Includes the entire scope of the domain model on every message

Benefits

1. No state management required
    1. Fewer bugs
2. Easy for consumers to publish fewer events via batching changes into one fact
3. Observability

Drawbacks

1. Individual event size is larger
2. Slower publishing if batching, more large events if not
3. Loss of intent

Fact Events

When and how can I use it?

When?

- Any time you want to know the current state of a domain
- Almost always, exception is when you only want to notify consumers of very specific events (e.g. “low inventory alert”)

How?

- Publish the entire state of your domain instead of deltas

```
{
 "cart_id": 3125123,
 "item_map": [ (521,1), (923,3) ],
 "shipping": "express"
}
```

Cart Fact Event

```
{
 "cart_id": 3125123,
 "item_id": 521,
 "quantity": 1
}
```

item_added_to_cart Delta Event

Streams record history

“The sequence of moves”

Tables represent state

“The state of the board”

Hashing

- Define which partition a message goes to via hash function and modulo
- Default is to use the key
- Can define a custom partitioning strategy by hashing on whatever values you want

Hashing

When and how can I use it?

When?

- Rarely
- Grouping for deduping or fresh filtering by something other than the key, like a subset of a composite key

How?

- Class that implements `Partitioner` and overrides the `partition()` method
- `ProducerConfig.PARTITIONER_CLASS_CONFIG= YourCustomPartitioner`

```
class CustomPartitioner : Partitioner {  1 Usage
    override fun partition(
        topic: String,
        keyObj: Any?,
        keyBytes: ByteArray?,
        value: Any?,
        valueBytes: ByteArray?,
        cluster: Cluster
    ): Int {
        if (keyObj is String) {
            return Utils.murmur2( data = keyObj.split( ...delimiters = ":").last().toByteArray()) %
                    cluster.partitionCountForTopic(topic)
        } else {
            throw Exception("Invalid key")
        }
    }
```

Compaction

- A type of cleanup policy
- Alternative to typical “delete” retention
- Can be combined with “delete” retention

Compaction

Benefits

1. Latest state
2. Fast consumption and recovery
3. Reduced storage

Drawbacks

1. Eventually consistent
2. No historical audit trail or replay ability
3. More CPU and memory usage on the broker
4. Careful key design
5. Offline consumers can miss tombstones

Compaction

When and how can I use it?

When?

- When consumers only care about the latest value for a key (e.g. fact topics)
- Finite unique keys (10s to 100s of millions limit)
- Compact + delete if values are time-sensitive and can expire
    - You can set up a periodic repopulation if needed

How?

- `cleanup.policy = compact` config value
- Tweak `max.compaction.lag.ms` and `segment.ms` to run often but not too often (24 hours and 12 hours respectively)
- Don't tweak min.cleanable.dirty.ratio because it causes compaction to run too often
- Send tombstones (null value) to delete messages

Batching and Timeouts

Batching while consuming

- Consume data within a bounded timeframe or size
- `List<Message>` instead of a single `Message`

Batching while producing

- Publish data within a bounded timeframe or size
- Publish a `List<Message>` instead of a single `Message`

Timeouts

- Client waits for the broker to respond
- Broker waits for the consumer to respond
- Max amount of time a consumer has to process a message/batch

Batching and Timeouts

Benefits

1. Reduce number of network requests
2. Increase throughput
3. Minimize load on the broker
4. Compression efficiency

Drawbacks

1. Introduces a small latency
2. Doesn’t inherently improve the performance of individual message processing
3. Increases rebalancing

Batching and Timeouts

When and how can I use it?

When?

- Almost always
- Exception is extremely low latency requirements where messages must be published or processed *immediately*

How?

- Producer
  - `batch.size`
  - `linger.ms`
  - `request.timeout.ms`
  - `delivery.timeout.ms`
- Consumer
  - `max.poll.records`
  - `max.poll.interval.ms`
  - `fetch.min.bytes`
  - `fetch.max.wait.ms`
  - `request.timeout.ms`
  - `session.timeout.ms`

```kotlin
fun main() {
    val props = Properties().apply {
        put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092")
        put(ConsumerConfig.GROUP_ID_CONFIG, "local")
        put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer::class.java.name)
        put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer::class.java.name)
        put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest")

        // Batching
        put(ConsumerConfig.MAX_POLL_RECORDS_CONFIG, 500)   // Poll up to 500 records
        put(ConsumerConfig.FETCH_MIN_BYTES_CONFIG, 10240)  // Wait for at least 10KB of data
        put(ConsumerConfig.FETCH_MAX_WAIT_MS_CONFIG, 1000) // Wait up to 1s for that data

        // Timeout
        put(ConsumerConfig.MAX_POLL_INTERVAL_MS_CONFIG, 300000) // Max 5 min between polls
        put(ConsumerConfig.SESSION_TIMEOUT_MS_CONFIG, 10000)    // 10s session timeout
        put(ConsumerConfig.REQUEST_TIMEOUT_MS_CONFIG, 15000)    // 15s broker response timeout
    }

    val consumer = KafkaConsumer<String, String>( properties = props)
    consumer.subscribe( topics = listOf("test-topic"))
    println("Started Kafka consumer...")
    while (true) {
        val records = consumer.poll( timeout = Duration.ofMillis( millis = 1000))
        for (record in records) {
            println("Received message: ${record.value()} from partition: ${record.partition()} offset: ${record.offset()}")
        }
    }
}
```

Deduplication

Exact duplicate messages

- Caused by network interruptions, rebalancing, etc…
- Idempotence in producer and consumer

Multiple different messages in a batch with the same key

- Happens even with log compaction enabled
- Process the latest message only and skip any others
- Much harder when you mix partitions (don’t mix partitions)
- Don’t dedupe tombstones

Deduplication

Benefits

1. Simple optimization strategy
2. Reduce number of network requests
3. Increase throughput
4. Minimize load on the broker
5. Preserves ordering

Drawbacks

1. Only works within a single poll
2. Not durable across application restarts or crashes
3. Requires carefully constructed data (compaction, fact topic, no side effects, sensible keys)
4. Tombstone edge cases, they must be ignored if they aren’t the last update

Deduplication

When and how can I use it?

When?

- Only when reading from fact topics
- Ideally when reading from log compacted topics
- Referentially transparent processing
- Processing incurs heavy cost (network calls, DB queries, etc...)
- Downstream systems aren't idempotent

How?

- Group by the last event for each key
- Linked hash map for a more performant mutable option

```
consumer.poll( timeout = Duration.ofMillis(500))
    .groupBy { it.key() }
    .map { (_, recordsForKey) -> recordsForKey.last() }
    .forEach {
        println("Processing record with key ${it.key()}")
    }
```

```
val records: ConsumerRecords<String, String> =
    consumer.poll( timeout = Duration.ofMillis(500))

val latestByKey = LinkedHashMap<String, String>()

for (record in records) {
    latestByKey[record.key()] = record.value()
}

for ((key, _) in latestByKey) {
    println("Processing record with key $key")
}
```

Fresh Filtering

- Like deduplication, but based on something in the value instead of the key
- Ignore messages where the field(s) inside the value that you care about aren’t changed
- Can be done in memory inside a single batch, but is typically done across multiple polls and partitions via state management

Fresh Filtering

Benefits

1. Correctness across restarts or crashes
2. Works across polls, instances, and partitions
3. Easy to implement a retention policy

Drawbacks

1. Requires some kind of durable state
2. Latency and throughput cost for state lookup
3. Storage overhead
4. Ordering matters and must be accounted for

Fresh Filtering

When and how can I use it?

<u>When?</u>

- Frequent updates to fields you don't care about or infrequent updates to fields you do
- You are the source of truth for the data, or want to guarantee correctness

<u>How?</u>

- Topic filtering (loader), a service with a consumer/producer pair that only publishes the data you care about to an internal topic
- Durable DB or cache like Postgres, MongoDB, or Redis. Only upsert when incoming data you care about has changed
- Embedded DB like RocksDB (via Kafka streams or otherwise)
- In memory with hash maps or other data structures

Draining

- Consume all the messages on the entire topic faster than anything can produce to it
- Result is guaranteed 0 lag
- Kafka as a persistent store
  - Requires compaction or no retention
- Send messages into state management
  - External DB or cache (often too slow)
  - Embedded DB
  - In memory

Draining

Benefits

1. Low latency
2. Plays nice with parallelism for high throughput
3. Lightweight and cheap
4. Resolves backpressure

Drawbacks

1. Requires state management and rebuilding on startup
2. Must be careful about always sending related events to the same partition / instance to maintain ordering

Draining

When and how can I use it?

When?

- Low latency required
- Reading from compacted fact topics makes it a lot easier
- The application is not the source of truth, eventually consistent data is okay
- Data set is not too large (big data)

How?

- Class that implements `Partitioner` and overrides the `partition()` method
- `ProducerConfig.PARTITIONER_CLASS_CONFIG= YourCustomPartitioner`

```
RocksDB.loadLibrary()
val dbDir = File( pathname = "rocksdb-example")
dbDir.mkdirs()
val options = Options().setCreateIfMissing(true)
val db = RocksDB.open( p0 = options,  p1 = dbDir.absolutePath)
consumer.poll( timeout = Duration.ZERO)
while (true) {
    val records = consumer.poll( timeout = Duration.ofMillis(1000))
    for (record in records) {
        try {
            db.put(
                p0 = record.key().toByteArray( charset = StandardCharsets.UTF_8),
                p1 = record.value().toByteArray( charset = StandardCharsets.UTF_8)
            )
    } catch (e: RocksDBException) {
            println("Error accessing RocksDB: ${e.message}")
        }
    }
}
```

Caching

- External caching
  - For kafka-backed embedded or in-memory caching see “Draining”
- Real-time data access and state management
- Cache prefetching
- Data synchronization

Caching

Benefits

1. Low latency external read access across systems
2. High QPS
3. Easy state management with idempotency
4. Persistent
5. Independent scaling

Drawbacks

1. Memory cost
2. Eventually consistent
3. More complexity

Caching

When and how can I use it?

When?

- Multiple services need fast access to the same data
- Specialized data structures (sorted sets, pub/sub, etc...)
- Small enough state to fit in memory

How?

- Publish the entire state of your domain instead of deltas

```
val jedis = Jedis( host = "localhost",  port = 6379)
consumer.poll( timeout = Duration.ZERO)
while (true) {
    val records = consumer.poll( timeout = Duration.ofMillis(1000))
    for (record in records) {
        try {
            jedis.set(
                record.key(),
                record.value()
            )
        } catch (e: Exception) {
            println("Error writing to Redis: ${e.message}")
        }
    }
}
```

Concurrency

- Kafka guarantees order within a partition
  - Max concurrency = # of partitions
  - Not thread safe
- Frameworks to handle async I/O calls, data transformations, and backpressure (LMAX Disruptor, Kotlin Flows)
  - Simple parallelism (thread pools) breaks ordering
  - Allows for custom configurations after single threaded Kafka polling is run (e.g. “step A must finish before step B, but all other steps can be run in parallel”)

Concurrency

Benefits

1. High throughput
2. Low latency
3. Mechanical sympathy and resource utilization
4. Clear, composable pipelines
5. Backpressure awareness

Drawbacks

1. More complexity
2. Operational tuning and configuration
3. Extra work to enable observability

Concurrency

When and how can I use it?

When?

- Heavy async I/O operations (HTTP API calls, DB queries, etc...)
- Very strict latency SLAs
- High volume and frequency of events

How?

- Publish the entire state of your domain instead of deltas

```
val kafkaMessages = channelFlow {
    records.forEachIndexed { index, record ->
        val key = record.key()
        val body = record.value()
        val meta = InputMetadata(
            topic = record.topic(),
            key = key,
            partitionOffset = Pair(record.partition(), record.offset()),
            isEndOfBatch = index + 1 == batchSize
        )
        send(element = meta to Payload(key = key, body = body))
    }
}.flowOn(context = consumerThread)

kafkaMessages.parallel(workers = config.workerConfig) { step ->
    pipeline(step)
}.collect { processedStep ->
    try {
        commitStrategy(processedStep, consumer)
    } catch (e: Throwable) {
        onError(e)
    }
}
```

Compression

- Reduces the size of messages
- Producer’s responsibility to compression batches per partition
- Broker periodically runs decompression to do things like compaction and validation
- Consumer’s responsibility to decompress messages

Compression

Benefits

1. Better disk utilization
2. Reduced network bandwidth
3. Higher throughput
4. Faster replication and compaction

Drawbacks

1. More CPU usage as broker processes
2. Extra work for consumers to decompress
3. Harder to work with encrypted data
4. Can add latency
5. Diminishing returns

Compression

When and how can I use it?

<u>When?</u>

- Most of the time, exception is extremely low latency systems with small messages
- Large messages
- High volume and frequency of events
- Large batch sizes

<u>How?</u>

- `compression.type` config value
  - Typically use lz4, tweak if needed
  - Almost never use gzip

Sources

1. Batching and Timeouts
2. Deduplication
3. Fresh Filtering
4. Hashing
    1. https://www.confluent.io/learn/kafka-partition-key/
    2. https://www.naleid.com/2021/05/02/kafka-topic-partitioning-replication.html
5. Draining
    1. https://www.confluent.io/blog/okay-store-data-apache-kafka/
6. Fact Events
    1. https://developer.confluent.io/courses/event-design/fact-vs-delta-events/
7. Caching
    1. https://redis.io/compare/redis-enterprise-and-kafka/
8. Concurrency
    1. https://github.com/benganglipson/pulse
    2. https://medium.com/@chetanshingare2991/kotlin-flows-a-complete-guide-to-asynchronous-streams-in-android-e790a6a9e116
    3. https://lmax-exchange.github.io/disruptor/
9. Compression
    1. https://www.confluent.io/blog/apache-kafka-message-compression/
    2. https://www.naleid.com/2021/05/02/kafka-topic-partitioning-replication.html
10. Schema Validation
    1. https://docs.confluent.io/platform/current/schema-registry/schema-validation.html#enabling-sv-on-a-topic-from-the-confluent-cli
11. Provenance
    1. https://www.ibm.com/think/topics/data-provenance
12. Compaction
    1. https://www.naleid.com/2023/07/30/understanding-kafka-compaction.html